Installing Libraries, NumPy, and Matplotlib

Python's standard library covers a lot of ground, but for specialized tasks — like fast numeric computation or making plots — you'll reach for external packages instead. In this activity you'll install two of the most widely used scientific Python libraries, use NumPy to work with arrays of numbers, and use Matplotlib to turn those numbers into a picture.

Content Learning Objectives

After completing this activity, students should be able to:

- Install an external Python package with pip and confirm that it is available.
- Create NumPy arrays and perform element-wise arithmetic on them.
- Generate a Matplotlib line plot with labeled axes.
- Combine NumPy and Matplotlib to visualize a mathematical function.

Process Skill Goals

During the activity, students should make progress toward:

- Reading and interpreting an error message when a package or import is missing. (Information Processing)
- Following a multi-step technical procedure as a team, and confirming each step before moving on. (Teamwork)

Facilitation Notes

Before class, make sure each team's Thonny (or environment) can actually reach the internet to install packages — installing during class can eat up a lot of time if the network is slow or blocked. If that's a concern, consider pre-installing `numpy` and `matplotlib` on lab machines beforehand and letting Model 1 be a confirmation-and-discussion exercise rather than a live install.

Model 1 focuses on the mechanics of installing something with pip and confirming success — for many students this is the first package they've ever installed themselves. Watch for teams confusing the *package name* typed into `pip install` with the *import name* typed into their code. Today the two happen to match (`numpy`, `matplotlib`), so use Question 7 to explicitly flag that this isn't always true (classic example: `pip install pillow` but `import PIL`).

Model 2 introduces arrays. The key misconception to watch for: students expect `*` on an array to behave like `*` on a list (repetition/concatenation) rather than element-wise multiplication. The table is built specifically to surface that contrast side by side — don't let a team skip past Question 2 without noticing it.

Model 3 builds a plot. Remind students that nothing appears until `plt.show()` runs — a common early mistake is assuming the plotting calls themselves display something. If a team's environment doesn't pop up a plot window, `plt.savefig("plot.png")` is a good fallback; it's worth demonstrating once on the projector. The exponential-decay curve in the last table row is a deliberate setup for the RC-circuit plotting exercise coming later this week.

Model 1 Installing Libraries

Some tasks call for code too specialized to ship with Python itself. NumPy (fast numeric arrays) and Matplotlib (plotting) are two of the most popular such packages, but neither one is built in the way `turtle` or `random` are — you have to install them yourself, once per environment.

Command (typed in a terminal, or the Thonny Shell)	What Happens
`pip list`	`Prints every package currently installed — probably a short list, without numpy or matplotlib yet`
`import numpy (before installing)`	`ModuleNotFoundError: No module named 'numpy'`
`pip install numpy`	`Downloads and installs numpy; ends with a line like Successfully installed numpy-2.1.0`
`import numpy (after installing)`	`(no output at all — a successful import is silent)`
`pip list (again)`	`numpy now appears somewhere in the list`

Do not install anything yet! Answer the first few questions before you touch the keyboard.

Questions (15 min)

Start time: _____

1. Based on the table above, what type of error do you get when you try to `import` a library that hasn't been installed yet? ModuleNotFoundError

2. What is the command to install the numpy library from a terminal? pip install numpy

3. In Thonny, what menu option lets you install a package without typing a pip command at all? Tools menu, Manage Packages

Now actually install `numpy` and `matplotlib` (using either method above), and confirm both show up in `pip list`. Ask for help if you get stuck!

4. After installing, how can you confirm a library is installed without writing a single line of Python code?

Run pip list (or open Tools -> Manage Packages in Thonny) and look for the package name in the list.

5. Type `import numpy` in the Shell. What output do you see? None -- a successful import produces no output at all.

6. Consider the command `pip install numpy` and the statement `import numpy`.

- **a)** What is the name of the library as given to pip (the *package* name)? numpy
- **b)** What is the name used in the `import` statement (the *import* name)? numpy
- **c)** In this case, are the two names the same? Yes

7. Not every package's pip name matches its import name — for example, you run `pip install pillow` but then write `import PIL`. Give one reason a library's install name and import name might end up different.

Answers will vary -- e.g. the obvious name was already taken on PyPI, the project was renamed at some point, or the developers chose a shorter/different name for typing in code than for the published package.

8. A classmate says "I don't need to install numpy — I already have Python installed." Explain what's wrong with this reasoning.

Python's built-in modules and separately-installed third-party packages are different things. Having Python installed doesn't include every library anyone has ever written -- numpy has to be installed on top of Python, once per environment.

Model 2 NumPy Basics

NumPy's core object is the **array** — a grid of numbers you can do math on all at once, instead of one number at a time in a loop. The conventional way to import it gives it a short alias:

```
1 import numpy as np
```

Python code	Shell output
`values = [10, 20, 30]`	`(no output)`
`values * 2`	`[10, 20, 30, 10, 20, 30]`
`arr = np.array([10, 20, 30])`	`(no output)`
`arr * 2`	`[20 40 60]`
`arr + 1`	`[11 21 31]`
`arr.sum()`	`60`
`arr.mean()`	`20.0`
`arr.max()`	`30`
`np.arange(0, 10, 2)`	`[0 2 4 6 8]`
`np.linspace(0, 1, 5)`	`[0. 0.25 0.5 0.75 1.]`

Do not type anything yet! Read the questions first.

Questions (20 min)

Start time: _____

9. What is the conventional alias used when importing numpy (as in `import numpy as ___`)? np
Now open the Shell, type each line of Model 2 one at a time, and record the output above (in the table). Place an asterisk next to anything you didn't expect.

10. Compare rows 1–2 with rows 3–4 above (the plain Python list versus the NumPy array). What is the difference between what `* 2` does to a list and what it does to an array?

Multiplying a list by 2 repeats/concatenates its contents (list * 2 makes a longer list); multiplying a NumPy array by 2 multiplies every individual element by 2 instead.

11. Based on the table, list three NumPy array methods (called like `arr.method()`) that summarize the values in an array. sum(), mean(), and max() (min() also works)

12. `np.arange(0, 10, 2)` takes three arguments. What do the first, second, and third arguments represent, in order? start, stop, step

13. How is `np.linspace` different from `np.arange`?

linspace's third argument is the count of evenly spaced points to generate between start and stop (inclusive), while arange's third argument is a step size and doesn't include the stop value.

14. Predict the output of `np.arange(1, 5)` (only two arguments given). [1 2 3 4]
Test your prediction in the Shell — were you right?

15. Predict the output of `arr / 0`, where `arr = np.array([1, 2, 3])`. Then test it in the Shell. What's different from what plain Python does with `1 / 0`?

Plain Python's 1 / 0 raises a ZeroDivisionError and stops the program. NumPy instead prints a RuntimeWarning and returns an array of inf (and/or nan) values, element by element, without crashing.

16. Why might it be useful that array operations don't require you to write your own loop?

Less code to write, less room for indexing mistakes, and NumPy's own loop (written in optimized C) runs much faster than an equivalent Python for loop.

Model 3 Matplotlib Basics

Matplotlib turns arrays of numbers into pictures. Its most commonly used piece is `pyplot`, almost always imported as `plt`.

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 10, 100)
y = np.sin(x)

plt.plot(x, y)
plt.xlabel("x")
plt.ylabel("sin(x)")
plt.title("My First Plot")
plt.show()
```

Create a new file `plot1.py`, enter the code above, and run it.

Questions (10 min)

Start time: _____

17. What appeared when you ran the program?

A window opened showing a smooth wavy curve (one full-ish period of a sine wave) running from x = 0 to x = 10.

18. Which single line of code is responsible for actually opening the plot window? plt.show()

19. Which two lines create the arrays of numbers that get plotted? The x = np.linspace(...) line and the y = np.sin(x) line.

20. What are the two arguments to `plt.plot()` in this example, and what does each one represent?

x and y -- the arrays of numbers for the horizontal and vertical axis, matched up point by point.

Now delete (or comment out) the three lines starting with `plt.xlabel`, `plt.ylabel`, and `plt.title`, and run the program again.

21. What does the plot look like now — what's missing compared to before?

The same curve shape appears, but with no axis labels or title -- just a bare line, which makes it much harder to tell what the numbers actually mean.

22. Why is it good practice to always label your axes?

A plot without labels doesn't tell a viewer what the numbers or units represent, which makes the plot hard to interpret or trust, even if the shape itself is correct.

Now change the line `y = np.sin(x)` to each of the following in turn, re-running each time, and describe the resulting shape.

Code for `y`	Shape of the plot
`y = x`	`A straight diagonal line through the origin.`
`y = x ** 2`	`An upward-curving parabola.`
`y = np.exp(-x)`	`Starts high near x = 0 and decays smoothly down toward 0 as x increases.`

23. Based on the last row of the table, describe in your own words what `np.exp(-x)` does to the values as `x` increases.

The values shrink toward zero as x grows -- this is exponential decay.

24. If you wanted two different curves — say `np.sin(x)` and `np.cos(x)` — on the *same* plot, how many times would you need to call `plt.plot()` before calling `plt.show()`? Twice -- once per curve, both before the single plt.show() call.